

Heaven's light is our guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



Lab Report

Course Code : CSE-3206
Course Name : Software Engineering Sessional
Experiment Name : Software Design Patterns
Date of Submission : 19.08.2026

Submitted by	Submitted to
Name: Jayed Hasan Roni, Md. Rafiul Islam Ovi, Md. Abdus Salam Roll: 2203073, 2203074, 2203075 Section: B Series: 22	Emrana Kabir Hashi Assistant Professor Department of Computer Science & Engineering, Ruet

Lab Report: Flyweight Pattern

1. Pattern Name

Flyweight Pattern

2. Category

Structural Design Pattern

3. Intent

The Flyweight Pattern minimizes memory usage and storage overhead by sharing as much data as possible with similar objects. It splits an object's state into intrinsic (invariant/shared) and extrinsic (variant/context-dependent) components to support large numbers of fine-grained objects efficiently

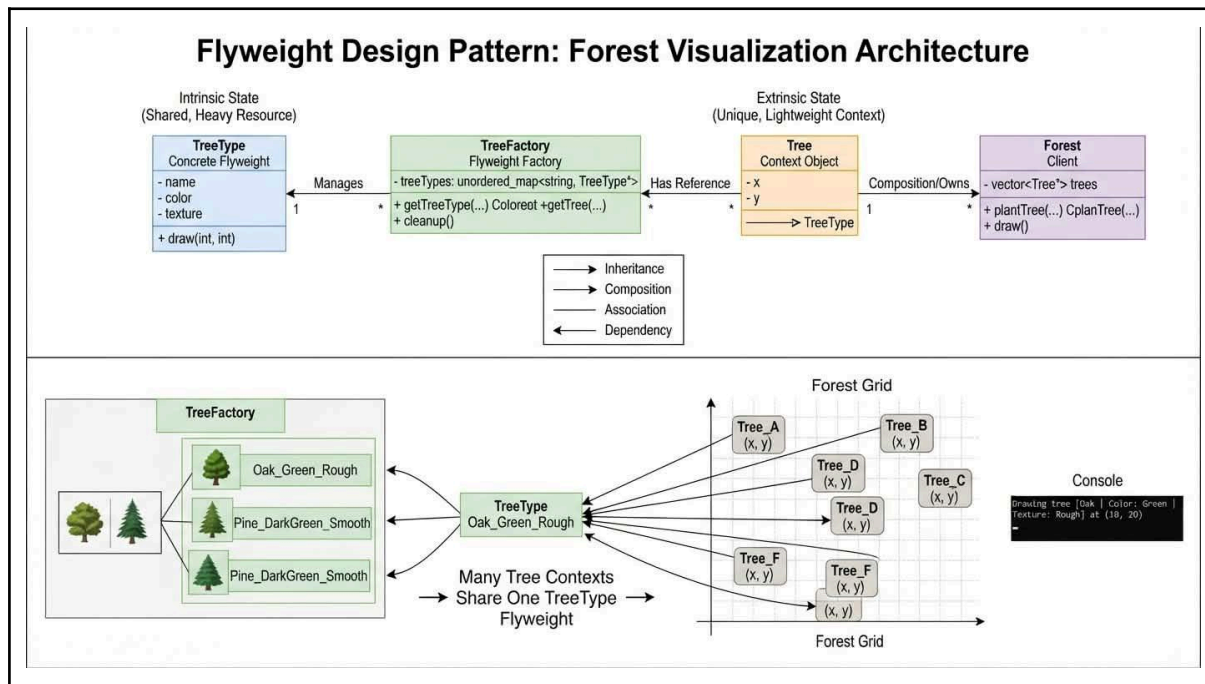
4. Problem Statement

Imagine developing a forest rendering feature for a video game or simulation that needs to display 1,000,000 tree objects. Each tree has visual properties like texture, color, and mesh (requiring heavy memory), as well as individual properties like (x, y) coordinates and height. Instantiating 1,000,000 individual Tree objects with heavy texture and mesh data will cause a severe memory shortage and crash the application.

5. Motivation

Creating a new instance for every single entity when large amounts of data are shared across instances leads to excessive memory consumption. The Flyweight pattern extracts shared, heavy, and immutable data (**Intrinsic State**) into dedicated shared objects, passing unique parameters (**Extrinsic State**) dynamically when methods are executed.

6. Pattern Structure (Conceptual)



7. Class Responsibilities

- **TreeType (Concrete Flyweight):** Holds heavy intrinsic properties shared across trees (e.g., name, color, texture).
- **TreeFactory (Flyweight Factory):** Manages a cache/dictionary of TreeType objects to avoid duplicating existing tree types.
- **Tree (Context / Client Object):** Contains unique extrinsic coordinates (x, y) and a reference to its shared TreeType.
- **Forest (Client):** Contains and renders all trees on the canvas.

8. Code Implementation (C++)

```
#include <iostream>
#include <string>
#include <unordered_map>
#include <vector>
using namespace std;

// 1. Concrete Flyweight (Stores Intrinsic State)
class TreeType {
private:
    string name;
    string color;
    string texture; // Heavy resource data representation
```

```

public:
    TreeType(string n, string c, string t) : name(n), color(c), texture(t) {}
    void draw(int x, int y) {
        cout << "Drawing tree [" << name << " | Color: " << color
            << " | Texture: " << texture << "] at coordinates (" << x << ", " << y << ")\n";
    }
};

// 2. Flyweight Factory
class TreeFactory {
private:
    static unordered_map<string, TreeType*> treeTypes;
public:
    static TreeType* getTreeType(string name, string color, string texture) {
        string key = name + "_" + color + "_" + texture;
        if (treeTypes.find(key) == treeTypes.end()) {
            treeTypes[key] = new TreeType(name, color, texture);
            cout << "--> [Factory] Creating NEW shared TreeType: " << key << "\n";
        } else {
            cout << "--> [Factory] Reusing EXISTING shared TreeType: " << key << "\n";
        }
        return treeTypes[key];
    }
    static void cleanup() {
        for (auto pair : treeTypes) {
            delete pair.second;
        }
        treeTypes.clear();
    }
};

// Initialize static member
unordered_map<string, TreeType*> TreeFactory::treeTypes;

// 3. Context Object (Contains Extrinsic State + Reference to Flyweight)
class Tree {
private:
    int x;
    int y;
    TreeType* type; // Shared intrinsic flyweight reference
public:
    Tree(int xCoord, int yCoord, TreeType* treeType) : x(xCoord), y(yCoord), type(treeType) {}
    void draw() {
        type->draw(x, y);
    }
};

// 4. Forest Client
class Forest {
private:
    vector<Tree*> trees;
public:
    void plantTree(int x, int y, string name, string color, string texture) {

```

```

    TreeType* type = TreeFactory::getTreeType(name, color, texture);
    Tree* tree = new Tree(x, y, type);
    trees.push_back(tree);
}
void draw() {
    for (auto tree : trees) {
        tree->draw();
    }
}
~Forest() {
    for (auto tree : trees) {
        delete tree;
    }
}
};

```

9. Execution Flow / Demonstration

```

int main() {
    Forest forest;
    cout << "--- Planting Trees in Forest ---\n";

    // Reuse intrinsic 'Oak' type
    forest.plantTree(10, 20, "Oak", "Green", "OakTexture.png");
    forest.plantTree(15, 25, "Oak", "Green", "OakTexture.png");

    // Reuse intrinsic 'Pine' type
    forest.plantTree(50, 60, "Pine", "Dark Green", "PineTexture.png");
    forest.plantTree(55, 65, "Pine", "Dark Green", "PineTexture.png");

    cout << "\n--- Rendering Forest ---\n";
    forest.draw();
    TreeFactory::cleanup();

    return 0;
}

```

10. Advantages

- **Massive Memory Reduction:** Avoids creating duplicate instances of high-cost data across thousands of objects.
- **Centralized Resource Management:** Heavy data assets are cached and managed in a central factory.
- **Scalability:** Allows applications to handle vast quantities of dynamic objects smoothly without running out of RAM.

11. Limitations

- **Increased Complexity:** Introduces additional code complexity by decoupling intrinsic and extrinsic states.
- **CPU Overhead:** Performing lookup calculations or passing extrinsic state parameters dynamically at runtime can introduce slight CPU trade-offs for memory savings.

12. Real-life Applications

- **Text Editors (Word Processors):** Each character in a document shares glyph texture and font data while maintaining unique coordinate positions.
- **Map / GIS Applications:** Rendering thousands of map icons (e.g., gas stations, restaurants) sharing icon graphic objects.

13. Industry Examples

- **Game Engines (e.g., Unity, Unreal):** Particle systems and terrain vegetation instantiation using shared mesh/material instances.
- **Java String Pool:** Java's `String` class reuses string literals from a pool rather than creating new objects for identical text content.

14. Sample Output

```
(base) md.rafiulislamovi@Mds-MacBook-Air-2 Lab_3 % cd "/Users/md.rafiulislamovi/Desktop/RUET_PDF/Semester-3.2/cse_3206_SE_lab/Lab_3/" && g++ -std=c++17 Flyweight.cpp -o Flyweight && "/Users/md.rafiulislamovi/Desktop/RUET_PDF/Semester-3.2/cse_3206_SE_lab/Lab_3/"Flyweight
--- Planting Trees in Forest ---
--> [Factory] Creating NEW shared TreeType: Oak_Green_OakTexture.png
--> [Factory] Reusing EXISTING shared TreeType: Oak_Green_OakTexture.png
--> [Factory] Creating NEW shared TreeType: Pine_Dark_Green_PineTexture.png
--> [Factory] Reusing EXISTING shared TreeType: Pine_Dark_Green_PineTexture.png

--- Rendering Forest ---
Drawing tree [Oak | Color: Green | Texture: OakTexture.png] at coordinates (10, 20)
Drawing tree [Oak | Color: Green | Texture: OakTexture.png] at coordinates (15, 25)
Drawing tree [Pine | Color: Dark Green | Texture: PineTexture.png] at coordinates (50, 60)
Drawing tree [Pine | Color: Dark Green | Texture: PineTexture.png] at coordinates (55, 65)
(base) md.rafiulislamovi@Mds-MacBook-Air-2 Lab_3 %
```

15. Conclusion

The Flyweight Pattern drastically reduces memory footprint by sharing common object state across vast numbers of items. By separating static intrinsic properties from dynamic extrinsic variables, systems remain lightweight and highly performant.